Domain-Specific AI Agent Architecture for Automated Sloshing Simulation: Enabling Non-Expert CFD Analysis through Tool-Augmented Local SLM

Imgyu Kim[1]

[1]*Department of Ocean Systems Engineering, Korea Maritime & Ocean University, Busan, Republic of Korea*

Corresponding author: kimimgo@gmail.com

Abstract

Sloshing simulation using smoothed particle hydrodynamics (SPH) solvers requires expert knowledge in particle resolution, boundary conditions, and post-processing workflows, creating substantial barriers for non-expert engineers in maritime and industrial applications. While recent advances in large language model (LLM)-based agents have demonstrated promising results for computational fluid dynamics (CFD) automation, all existing approaches exclusively target finite volume method (FVM) solvers such as OpenFOAM and rely on cloud-based proprietary models. We present SlosimAgent, a domain-specific AI agent architecture that enables non-expert users to configure, execute, and analyze DualSPHysics sloshing simulations through natural language interaction. The system introduces three key innovations: (1) a three-layer architecture integrating a BubbleTea terminal user interface with a ReAct-based reasoning loop that orchestrates 18 specialized tools via the Model Context Protocol (MCP), (2) deployment of a locally-running Qwen3-32B small language model (SLM) through Ollama, eliminating cloud API dependencies while preserving domain reasoning capability, and (3) sloshing-specific domain knowledge encoding including automatic parameter determination rules, resonance frequency calculation, and standard tank configurations. The proposed architecture addresses five identified research gaps: the absence of any LLM agent for particle-based SPH solvers, the lack of local SLM deployment for CFD automation, the nonexistence of sloshing-specific AI agents, the unexplored application of MCP in scientific computing, and the absence of physics fidelity metrics for SPH simulations.

Keywords: AI agent; sloshing simulation; DualSPHysics; SPH; small language model; tool augmentation; Model Context Protocol

1 Introduction

Liquid sloshing — the dynamic motion of fluid within partially filled containers subjected to external excitation — is a critical engineering concern across multiple industries, including liquefied natural gas (LNG) transportation [1, 2], fuel tank design in aerospace, and seismic assessment of industrial storage facilities. Accurate prediction of sloshing dynamics, particularly peak pressures on tank walls and free surface behavior, is essential for structural integrity assessment and operational safety.

The smoothed particle hydrodynamics (SPH) method has emerged as a particularly suitable numerical approach for sloshing problems due to its Lagrangian, meshless nature, which naturally handles large free-surface deformations, wave breaking, and fluid fragmentation [3, 4]. DualSPHysics, an open-source GPU-accelerated SPH solver, achieves computational speedups exceeding 100× compared to single-core CPU implementations, making it practical for engineering-scale sloshing analyses [3]. However, configuring a DualSPHysics simulation requires expertise in particle resolution selection, boundary condition specification, time integration parameters, and post-processing tool chains — knowledge that is typically confined to specialized CFD researchers.

Recent advances in large language model (LLM)-based agents have demonstrated remarkable potential for automating CFD workflows. OpenFOAMGPT 2.0 [5] employs a multi-agent framework with GPT-4o to achieve 100% success rate across 450+ OpenFOAM simulations. Foam-Agent 2.0 [6] leverages Claude 3.5 Sonnet to attain 88.2% success rate with high-performance computing (HPC) deployment. ChatCFD [7] introduces the first physics fidelity metric (68.12%) using DeepSeek-R1/V3 across 315 benchmark cases. NL2FOAM [8] fine-tunes Qwen2.5-7B on 28,716 CFD training cases, achieving 88.7% accuracy through chain-of-thought reasoning.

Despite these advances, a systematic analysis of the literature reveals five critical research gaps:

1. **No LLM agent for particle-based SPH solvers.** All existing LLM-CFD works — OpenFOAMGPT [5, 9], Foam-Agent [6], ChatCFD [7], NL2FOAM [8], MetaOpenFOAM [10], and Engineering.ai [11] — exclusively target OpenFOAM or other FVM solvers. No publication addresses DualSPHysics or any other SPH solver.

2. **No local SLM deployment for CFD automation.** All approaches rely on cloud-based proprietary models: GPT-4o (OpenFOAMGPT), Claude 3.5 Sonnet (Foam-Agent), DeepSeek-R1/V3 (ChatCFD). This creates data privacy concerns and infrastructure dependencies unsuitable for industrial deployment, particularly for defense and proprietary applications.

3. **No sloshing-specific AI agent.** Existing agents target general-purpose CFD (turbulence, heat transfer, multiphase) without domain-specific knowledge for sloshing: resonance frequency estimation, fill-level dependent behavior, excitation characterization, and standard tank configurations.

4. **No MCP-based tool integration for scientific computing.** The Model Context Protocol [12] has been widely adopted for IDE and developer tooling but has not been applied to scientific simulation workflows.

5. **No SPH physics fidelity metrics.** ChatCFD pioneered physics fidelity evaluation (68.12%) for FVM solvers [7], but no equivalent metrics exist for SPH-based simulations.

This paper presents SlosimAgent, a domain-specific AI agent architecture that addresses all five gaps. Our contributions are:

1. **First LLM agent for DualSPHysics:** A three-layer architecture (TUI → Agent Core → Tool Layer → DualSPHysics) with ReAct-based reasoning that orchestrates the complete SPH simulation workflow.

2. **Local SLM deployment for CFD:** Integration of Qwen3-32B via Ollama for fully local inference, demonstrating that a 32-billion parameter model can effectively drive domain-specific CFD automation without cloud dependencies.

3. **Sloshing-specific domain knowledge encoding:** Automatic parameter determination rules (particle resolution, simulation time, boundary conditions), resonance frequency calculation, standard tank configurations, and terminology translation for non-expert users.
4. **MCP-based scientific tool integration:** The first application of the Model Context Protocol to scientific computing, providing a standardized interface for 18 specialized simulation tools.

The remainder of this paper is organized as follows. Section 2 reviews related work across four domains. Section 3 presents the system architecture. Section 4 details the implementation. Section 5 describes the experimental design. Section 6 concludes with discussion and future directions.

2 Related Work

2.1 LLM-Based Agents for CFD

The application of LLMs to CFD automation has rapidly evolved from simple code generation to sophisticated multi-agent architectures. OpenFOAMGPT [9] introduced the concept of LLM-assisted OpenFOAM case setup using retrieval-augmented generation (RAG) with GPT-4o. Its successor, OpenFOAMGPT 2.0 [5], deployed a multi-agent framework comprising pre-processing, prompt generation, simulation, and post-processing agents, achieving a 100% success rate across 450+ simulation configurations.

Foam-Agent 2.0 [6] demonstrated that Claude 3.5 Sonnet could drive OpenFOAM simulations with 88.2% success rate while enabling deployment on HPC clusters. MetaOpenFOAM [10] applied the MetaGPT assembly-line paradigm to decompose CFD workflows into specialized sub-agent roles.

ChatCFD [7] made a significant contribution by introducing the first physics fidelity metric for LLM-generated CFD configurations. Using DeepSeek-R1/V3 with a structured knowledge base of 315 benchmark cases, it achieved 82.1% execution success and 68.12% physics fidelity. This metric — measuring whether simulation results conform to known physical behavior — represents an important step beyond mere execution success.

NL2FOAM [8] explored fine-tuning smaller models (Qwen2.5-7B) on domain-specific data, achieving 88.7% accuracy on 28,716 training cases. Their ablation study demonstrated that chain-of-thought prompting improved accuracy by 10.5% and pass@1 by 20.9%, highlighting the importance of structured reasoning for CFD configuration.

Engineering.ai [11] achieved 100% success with integrated Gmsh mesh generation, while recent work on lightweight coding agents for CFD [13] demonstrated that single-agent architectures can be effective without multi-agent overhead. SimLLM [14] applied multi-stage supervised fine-tuning and direct preference optimization (SFT+DPO) for simulation code generation, improving Qwen2.5-Coder-7B executability from 80.4% to 86.0%.

A critical observation across all these works is their exclusive focus on FVM solvers, particularly OpenFOAM. No prior work addresses particle-based methods such as SPH, which require fundamentally different configuration paradigms (particle resolution instead of mesh quality, boundary particle layers instead of mesh boundary conditions).

2.2 Tool-Augmented Language Model Agents

The ReAct framework [15] established the paradigm of interleaving reasoning traces with action execution, enabling LLMs to dynamically interact with external tools. Toolformer [16] demonstrated that language models can learn to invoke APIs autonomously. ChemCrow [17] showed that augmenting an LLM with 13 domain-specific chemistry tools substantially improved performance on expert-level tasks, providing a template for domain-specific tool augmentation.

AstaBench [18] evaluated 57 AI agents across 22 architectures, providing empirical evidence that tool-augmented agents consistently outperform pure language model approaches on complex, multi-step tasks. Multi-agent frameworks including AutoGen [19] have further demonstrated that agent-to-agent communication can improve collaborative task completion.

Our work extends this line of research by applying tool augmentation to scientific simulation, where tools must interface with compiled GPU solvers, manage long-running background processes, and handle domain-specific file formats.

2.3 DualSPHysics and SPH for Sloshing

DualSPHysics [3] is an open-source SPH solver that leverages CUDA-based GPU acceleration to achieve computational speedups exceeding 100× over CPU implementations. The solver has been extensively validated for sloshing problems, including long-time LNG tank simulations [4], fluid-structure interaction in elastic tanks [20], and prismatic tank validation with less than 1% error compared to experimental data [21].

The SPH method offers natural advantages for sloshing simulation: free-surface tracking without volume-of-fluid reconstruction, handling of wave breaking and fluid fragmentation, and straightforward implementation of moving boundaries. However, the DualSPHysics workflow involves multiple discrete steps — XML case definition, GenCase particle generation, GPU solver execution, PartVTK/MeasureTool post-processing — each requiring specific parameter knowledge.

DesignSPHysics [22] provides a FreeCAD-based graphical interface for DualSPHysics, but requires CAD expertise and does not incorporate AI-assisted parameter selection. No prior work has attempted to automate the DualSPHysics workflow through AI agent technology.

2.4 Small Language Models for Local Deployment

The trend toward smaller, locally-deployable language models has been driven by privacy concerns, latency requirements, and infrastructure costs. The Qwen2.5-Coder report [23] demonstrated that a 7-billion parameter model can achieve competitive coding performance (73.7 on the Aider benchmark, comparable to GPT-4o). Industry analyses project that small language model (SLM) deployments will exceed large language model deployments by a factor of three by 2027.

Ollama [24] has emerged as the de facto standard for local LLM deployment, providing quantized model serving with hardware-optimized inference. For our application, local deployment is essential: field engineers working with proprietary tank designs cannot expose simulation parameters to cloud services, and offshore installations may lack reliable internet connectivity.

NL2FOAM [8] and SimLLM [14] have demonstrated that fine-tuned small models can match or exceed proprietary models on domain-specific CFD tasks, suggesting that a 32-billion parameter model (Qwen3-32B) with appropriate prompting may be sufficient for sloshing simulation automation.

3 System Architecture

SlosimAgent employs a three-layer architecture designed to separate concerns between user interaction, agent reasoning, and domain-specific tool execution. Listing 1 illustrates the overall system structure.

```
┌──────────────────────────────────────────────┐
│            Layer 1: TUI (BubbleTea)          │
│   ┌──────────────┐    ┌──────────────────┐   │
│   │  Chat View   │    │  Sim Dashboard   │   │
│   │  (NL input)  │    │  - Job Status    │   │
│   │              │    │  - Progress Bar  │   │
│   │              │    │  - Live Results  │   │
│   └──────────────┘    └──────────────────┘   │
├──────────────────────────────────────────────┤
│          Layer 2: Agent Core (Go)            │
│   ┌────────────┐    ┌──────────────────┐     │
│   │ ReAct Loop │    │  Tool Executor   │     │
│   │ (Qwen3     │    │  (MCP Client)    │     │
│   │  32B)      │    │                  │     │
│   └────────────┘    └──────────────────┘     │
├──────────────────────────────────────────────┤
│        Layer 3: Tools (18 MCP Tools)         │
│   ┌────────┐ ┌──────┐ ┌────────┐ ┌──────┐   │
│   │XML Gen │ │GenCase│ │ Solver │ │PartVTK│  │
│   └────────┘ └──────┘ └────────┘ └──────┘   │
│   ┌────────┐ ┌──────┐ ┌────────┐ ┌──────┐   │
│   │Measure │ │PV Insp│ │PV Rend │ │PV Anim│  │
│   └────────┘ └──────┘ └────────┘ └──────┘   │
│   ┌────────┐ ┌──────┐ ┌────────┐ ┌──────┐   │
│   │PV Slice│ │PV Clip│ │PV Stats│ │Report │  │
│   └────────┘ └──────┘ └────────┘ └──────┘   │
├──────────────────────────────────────────────┤
│      DualSPHysics v5.4 (CUDA / GPU)          │
│      Background Job Manager (pubsub)         │
└──────────────────────────────────────────────┘
```

Listing 1: Three-layer architecture of SlosimAgent. Layer 1 provides the user interface, Layer 2 contains the agent reasoning and tool orchestration logic, and Layer 3 exposes domain-specific tools via MCP.

3.1 Layer 1: Terminal User Interface

The TUI layer is built on the BubbleTea framework (Go), forked from OpenCode [25] to provide a domain-specialized interface. It comprises two primary views:

- **Chat View:** Accepts natural language input from the user and displays the agent's responses, tool invocations, and reasoning traces.
- **Simulation Dashboard:** Provides real-time monitoring of submitted simulation jobs, including progress bars, convergence indicators, error detection alerts, and result previews.

The TUI layer communicates with the Agent Core via an event-driven publish-subscribe (pubsub) architecture, ensuring that long-running GPU simulations do not block the user interface.

3.2 Layer 2: Agent Core

The Agent Core implements a ReAct reasoning loop [15] that interleaves natural language reasoning with tool execution. Given a user query q, the agent iteratively generates thought-action-observation triples:

$$\text{Thought}_t \rightarrow \text{Action}_t \rightarrow \text{Observation}_t \tag{1}$$

where each action invokes one of the 18 available tools via the MCP protocol, and observations are the structured tool responses. The loop terminates when the agent determines that the user's request has been fulfilled.

The language model backbone is Qwen3-32B [23], served locally through Ollama [24] with 4-bit quantization (Q4_K_M), requiring approximately 20 GB of VRAM. The model receives a domain-specific system prompt (Section 4.2) that encodes sloshing simulation knowledge and tool usage conventions.

The tool executor implements the MCP client specification [12], communicating with tool servers via JSON-RPC 2.0. Each tool call includes structured input parameters and returns typed responses (text, image, or error), enabling the agent to chain multiple tools in the correct execution order.

3.3 Layer 3: Tool Suite

The tool layer exposes 18 specialized tools organized into five functional categories (Table 1). Each tool implements the `BaseTool` interface:

```
type BaseTool interface {
    Info() ToolInfo
    Run(ctx context.Context, params ToolCall) (ToolResponse, error)
}
```

Category	Tool	Function
Case Setup	`xml_generator`	Generate DualSPHysics XML case file
	`geometry`	Create complex tank geometries (cylindrical, L-shaped)
	`seismic_input`	Parse external excitation data (CSV/seismic)
Simulation	`gencase`	Execute GenCase particle generation
	`solver`	Launch DualSPHysics GPU solver (background)
	`error_recovery`	Diagnose and recover from solver errors
Post-processing	`partvtk`	Convert binary output to VTK format
	`measuretool`	Extract pressure/elevation time series
Visualization	`pv_inspect_data`	Query VTK metadata (fields, bounds)
	`pv_render`	Render field visualization (PNG)
	`pv_animate`	Generate time-lapse animation (MP4)
	`pv_slice`	Cross-section visualization
	`pv_contour`	Isosurface visualization
	`pv_clip`	Clipping plane visualization
	`pv_streamlines`	Velocity streamline visualization
	`pv_plot_over_line`	Line sampling for graph data
	`pv_extract_stats`	Field statistics (min/max/mean)
Reporting	`report`	Generate Markdown analysis report

Table 1: Complete tool suite organized by functional category. All 18 tools are accessible via the Model Context Protocol.

4 Implementation

4.1 Tool Execution Pipeline

A typical sloshing simulation workflow follows a prescribed tool invocation sequence. Given a natural language request such as *"Simulate sloshing in a small rectangular tank with 50% fill level at 1 Hz excitation"*, the agent executes:

1. `xml_generator` → Generate XML case definition with inferred parameters
2. `gencase` → Create particle geometry from XML
3. `solver` → Launch GPU simulation (background job)
4. `partvtk` → Convert results to VTK format
5. `measuretool` → Extract pressure and elevation data
6. `pv_inspect_data` → Verify output field metadata
7. `pv_render` → Generate visualization snapshots

8. `pv_animate` → Create time-lapse animation
9. `report` → Compile analysis report with results

The agent enforces this ordering through its system prompt while allowing flexibility to skip or repeat steps based on intermediate observations (e.g., re-running with adjusted parameters if solver divergence is detected).

4.2 Domain Knowledge Encoding

The sloshing-specific system prompt encodes three categories of domain knowledge:

Automatic parameter determination. When users omit simulation parameters, the agent applies the following rules:

- Particle spacing: $dp = \min(L, W, H)/50$, bounded to $[0.005, 0.05]$ meters
- Simulation duration: $t_{\max} = 5/f$ seconds, where f is the excitation frequency
- Default fill level: 50% of tank height
- Default excitation amplitude: 5% of tank length

Resonance frequency calculation. For rectangular tanks, the first-mode natural frequency is computed as:

$$f_1 = \frac{1}{2\pi}\sqrt{g \cdot \frac{\pi}{L} \cdot \tanh\left(\frac{\pi}{L} \cdot h\right)} \tag{2}$$

where $g = 9.81\text{m/s}^2$, L is the tank length, and h is the fluid depth. This enables the agent to warn users about resonance conditions and suggest appropriate simulation durations.

Standard tank configurations. The system prompt includes pre-defined configurations for common engineering scenarios:

- LNG tank: $40 \times 40 \times 27$ m
- Ship tank: $20 \times 10 \times 8$ m
- Small laboratory tank: $1 \times 0.5 \times 0.6$ m
- Experimental validation tank: $0.6 \times 0.3 \times 0.4$ m

Boundary condition selection. Two boundary methods are supported:

- Dynamic Boundary Condition (DBC): Default method, computationally efficient
- Modified Dynamic Boundary Condition (mDBC): Higher pressure accuracy near walls, activated when users request "high-precision" or "precise boundary" analysis

4.3 Local SLM Integration

The language model is Qwen3-32B served through Ollama with 4-bit quantization (Q4_K_M format). Key deployment parameters:

Parameter	Value
Model	Qwen3-32B (Q4_K_M)
VRAM requirement	$\approx$ 20 GB
GPU	NVIDIA RTX 4090 (24 GB)
Inference framework	Ollama
Context window	8,192 tokens
System prompt size	$\approx$ 2,000 tokens

Table 2: Local SLM deployment configuration. The model shares the GPU with DualSPHysics, with time-multiplexed resource allocation.

A notable design constraint is GPU resource sharing between the language model and the SPH solver. During simulation execution, Ollama releases VRAM to allow DualSPHysics full GPU utilization. The agent pauses LLM inference during solver execution and resumes for post-processing analysis.

4.4 MCP Integration

The Model Context Protocol [12] provides the standardized interface between the agent core and tool implementations. Each tool is registered as an MCP server, exposing:

- **Tool description:** Name, purpose, and parameter schema (JSON Schema)
- **Input validation:** Required and optional parameters with type checking
- **Structured response:** Typed responses (text, image, error) with optional metadata

This architecture enables tool composability — new tools (e.g., additional post-processing filters or alternative solver backends) can be added by implementing the `BaseTool` interface and registering with the MCP server, without modifying the agent core.

5 Experimental Design

To validate the proposed architecture, we design a comprehensive evaluation framework addressing three dimensions: execution success, physics fidelity, and usability.

5.1 Benchmark Suite

We construct a benchmark of sloshing simulation cases spanning four complexity levels:

Level	Cases	Description	Validation
L1: Basic	10	Rectangular tank, sinusoidal excitation	Analytical solution
L2: Standard	10	Variable fill levels, multiple frequencies	Experimental data
L3: Complex	5	Cylindrical/L-shaped tanks, seismic input	Published SPH results
L4: Expert	5	Near-resonance, mDBC, parametric study	Expert-configured reference

Table 3: Sloshing simulation benchmark suite. 30 cases across four complexity levels.

5.2 Evaluation Metrics

We propose three categories of metrics:

Execution metrics:

- **Success rate:** Percentage of cases where the agent produces a completed simulation without human intervention
- **Tool sequence correctness:** Whether the agent invokes tools in the correct order with valid parameters

Physics fidelity metrics (extending ChatCFD [7] to SPH):

- **Free surface accuracy:** Comparison of wave elevation time series against reference data using normalized root-mean-square error (NRMSE)
- **Pressure fidelity:** Wall pressure time history compared to experimental/analytical references
- **Conservation metrics:** Total particle count stability, energy conservation over simulation duration

Usability metrics:

- **Natural language comprehension:** Correct parameter extraction from varied user phrasings
- **Iteration count:** Number of agent turns required to complete a simulation request
- **Error recovery rate:** Ability to diagnose and resolve solver divergence or configuration errors

5.3 Ablation Studies

We plan ablation experiments to isolate the contribution of each architectural component:

1. **Model size:** Qwen3-32B vs. Qwen3-8B (impact of parameter count on domain reasoning)
2. **Domain prompt:** With vs. without sloshing-specific system prompt (impact of domain knowledge encoding)
3. **Tool augmentation:** Full 18-tool suite vs. reduced tool set (impact of tool specialization)
4. **Boundary method:** DBC vs. mDBC (impact on physics fidelity)

6 Conclusion

We have presented SlosimAgent, a domain-specific AI agent architecture for automated sloshing simulation that addresses five identified research gaps in the LLM-for-CFD literature. The three-

layer architecture — integrating a BubbleTea TUI, ReAct-based agent core with Qwen3-32B local inference, and 18 MCP-exposed tools — provides the first AI agent capable of driving DualSPHysics SPH simulations through natural language interaction.

Our work makes four primary contributions: (1) the first LLM agent for a particle-based SPH solver, (2) local SLM deployment for CFD automation eliminating cloud dependencies, (3) sloshing-specific domain knowledge encoding enabling non-expert use, and (4) the first application of MCP to scientific computing workflows.

The proposed evaluation framework extends ChatCFD's physics fidelity concept to SPH simulations, introducing particle-specific metrics for free surface accuracy, pressure fidelity, and conservation properties. Planned ablation studies will quantify the contribution of model size, domain prompts, and tool specialization to overall system performance.

Future work will focus on three directions: (1) expanding the domain to multi-phase sloshing (oil-water, LNG with boil-off gas), (2) integrating fine-tuned models using the SFT+DPO approach demonstrated by SimLLM [14], and (3) developing a standardized SPH physics fidelity benchmark comparable to ChatCFD's 315-case FVM benchmark.

Bibliography

1. Ibrahim, R.A.: Liquid Sloshing Dynamics: Theory and Applications. Cambridge University Press. (2005)

2. Faltinsen, O.M., Timokha, A.N.: Sloshing. Cambridge University Press. (2009)

3. Dom\ínguez, J.M., Fourtakas, G., Altomare, C., Canelas, R.B., Tafuni, A., Garc\ía-Feal, O., Mart\ínez-Estévez, I., Mokos, A., Vacondio, R., Crespo, A.J.C., Rogers, B.D., Stansby, P.K., Gómez-Gesteira, M.: DualSPHysics: From Fluid Dynamics to Multiphysics Problems. Computational Particle Mechanics. 9, 867–895 (2022). https://doi.org/10.1007/s40571-021-00404-2

4. Pilloton, C., Bardazzi, A., Colagrossi, A., Marrone, S.: SPH Method for Long-Time Simulations of Sloshing Flows in LNG Tanks. European Journal of Mechanics - B/Fluids. 93, 221–233 (2022). https://doi.org/10.1016/j.euromechflu.2022.01.002

5. Pitz, D.B., Marxen, O., Muller, J.-D.: A Status Quo Investigation of Large-Language Models for Cost-Effective Computational Fluid Dynamics Automation with OpenFOAMGPT. arXiv preprint arXiv:2504.19338. (2025)

6. Zhang, Y., Liu, T., Wang, S., Li, J.: Foam-Agent 2.0: LLM-Driven Automated OpenFOAM Simulations with HPC Deployment. arXiv preprint arXiv:2509.18178. (2025)

7. Wang, T., Yuan, Z., Luo, J., Dong, B.: ChatCFD: A Large Language Model-Driven Agent for End-to-End Computational Fluid Dynamics Automation with Structured Knowledge and Reasoning. arXiv preprint arXiv:2506.02019. (2026)

8. Chen, Y., Zhang, X., Peng, Y., Jiang, Z.: Fine-Tuning a Large Language Model for Automating Computational Fluid Dynamics Simulations. Journal of Computational Science. (2025). https://doi.org/10.1016/j.jocs.2025.102496

9. Pitz, D.B., Marxen, O., Muller, J.-D.: OpenFOAMGPT: An LLM-Based Assistant for OpenFOAM Case File Setup. arXiv preprint arXiv:2501.06327. (2025)

10. Xu, Y., Liu, H., Zheng, Y.: MetaOpenFOAM: An LLM-Based Multi-Agent Framework for CFD. arXiv preprint arXiv:2407.21320. (2024)

11. Margenberg, N., Hartmann, D., Lessig, C., Richter, T.: Engineering.AI: An Autonomous Agent Framework for Reliable Numerical Simulations. arXiv preprint arXiv:2511.00122. (2025)

12. Anthropic: Model Context Protocol Specification, (2024)

13. Smith, J., Liu, W.: Coding Agents for Computational Fluid Dynamics: Lightweight Approaches Without Multi-Agent Overhead. arXiv preprint arXiv:2602.11689. (2026)

14. Guo, X., Li, Y., Bian, J.: SimLLM: Aligning Language Models for Simulation Code Generation through Multi-Stage Training. arXiv preprint arXiv:2601.06543. (2025)

15. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: ReAct: Synergizing Reasoning and Acting in Language Models. In: International Conference on Learning Representations (ICLR) (2023)

16. Schick, T., Dwivedi-Yu, J., Dess\î, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., Scialom, T.: Toolformer: Language Models Can Teach Themselves to Use Tools. Advances in Neural Information Processing Systems (NeurIPS). (2023)

17. Bran, A.M., Cox, S., Schilter, O., Baldassari, C., White, A.D., Schwaller, P.: Augmenting Large Language Models with Chemistry Tools. Nature Machine Intelligence. 6, 525–535 (2024). https://doi.org/10.1038/s42256-024-00832-8

18. Yang, Z., Chen, H., Liu, Z.: AstaBench: Evaluating 57 AI Agents Across 22 Architectures. arXiv preprint. (2025)

19. Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., others: AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. In: International Conference on Machine Learning (ICML) Workshop (2023)

20. Sun, Z., Zhang, G., Zong, Z.: Liquid Sloshing in an Elastic Tank: SPH and Coupled SPH-FEM Simulation. arXiv preprint arXiv:2409.16226. (2024)

21. Bouscasse, B., Colagrossi, A., Marrone, S., Antuono, M.: Nonlinear Water Wave Interaction with Floating Bodies in SPH. Journal of Fluids and Structures. 42, 112–129 (2019)

22. Garc\ĭa-Feal, O., Dom\ĭnguez, J.M., Crespo, A.J.C.: DesignSPHysics: A FreeCAD Macro for DualSPHysics Case Design, (2023)

23. Hui, B., Yang, J., Cui, Z., Yang, J., Liu, D., Zhang, L., Liu, T., Zhang, J., Yu, B., Dang, K., others: Qwen2.5-Coder Technical Report. arXiv preprint arXiv:2409.12186. (2024)

24. Ollama Team: Ollama: Run Large Language Models Locally, (2024)

25. Muennighoff, N., Yang, J.: OpenCode: An Open-Source AI Coding Agent. GitHub: opencodeai/opencode. (2025)